

EC4219: Software Engineering (Spring 2025)

Homework 2: Verification of Partial Correctness

Due: 6/29, 14:00 (submit on GIST LMS)

Instructor: Sunbeom So

Important Notes

- **Evaluation criteria**

The correctness of your implementation will be evaluated using testcases:

$$\frac{\text{\#Passed}}{\text{\#Total}} \times 100$$

- “Total”: testcases prepared by the instructor (not disclosed before the evaluation).
- “Passed”: testcases where expected outputs match your outputs.

- **Compilable**

Make sure that your submission is successfully compiled. If your code cannot be compiled, you will get 0 points for the corresponding HW.

- **Academic Integrity**

Violating academic integrity will result in an F, even after the end of the semester.

[Click here to check the rules related to academic integrity.](#)

Be aware that proving your academic integrity is entirely your responsibility.

- **No Changes on Templates, File Names, and File Extensions**

Your job is to complete (* TODO *) parts in provided code templates. You should not modify the other templates. Do not change the file names. The submitted files should have .ml extensions, not the others (e.g., .pdf, .zip, .tar).

- **Regarding Using LLM-based Tools**

You can use LLM-based tools to complete your HW, as long as (1) you use LLMs by yourself, (2) you can reproduce all the steps assisted by LLMs, and (3) you can clearly explain all the details of your implementation (including LLM-generated parts).

Even though you claim that you have completed HW with the help of LLMs, if your submission is highly similar to other students' code and you cannot reproduce certain steps by any reason (including the nondeterminism of LLMs), I will consider that you committed academic misconduct.

1 Goal

Your goal is to implement a verifier for a simple imperative language. Specifically, your verifier should be able to correctly verify whether a given program satisfies specifications (pre- and postconditions, loop invariants, and assertions) or not.

2 Input-Output Examples

Consider the example program from [hw2/benchmark/array_if](#):

```

1 @pre {true}
2 @post {true}
3
4 array_if (int i) returns (int rv) {
5     int[10] a;
6
7     if (i > 9) {
8         assert(i >= 0 && i < len(a)); (* unsafe *)
9     }
10    else {
11        if ( i < 0 ) {
12            assert(i >= 0 && i < len(a)); (* unsafe *)
13        }
14        else { (* i>=0 && i <= 9 *)
15            assert(i >= 0 && i < len(a)); (* safe *)
16        }
17    }
18
19    return rv;
20 }
```

The intended meaning of each syntax element should be self-explanatory. Your verifier must be able to validate whether the given program is partially correct with respect to the function specification (@pre and @post) and the assertions. Specifically, if you run the command

```
$ ./main.native -input ../benchmark/array_if
```

you should obtain the following result:

```

1 === Program ===
2 @pre {(true /\ (forall [a]. len[a] >= 0))}
3 @post {true}
4 ...
5 [1] line 8 : potentially unsafe
6 [2] line 12 : potentially unsafe
7 [3] line 15 : safe
8 === Verification Result ===
9 true, 1
10 Time : ...
```

In the verification result (line 9), `true` indicates that the postcondition is satisfied (assuming the precondition holds), and 1 is the number of `assert` conditions proven to be safe. At line 2, `(forall [a]. len[a] >= 0)`, which states that every array's size is non-negative, is the “common” function precondition automatically inserted¹ by the `Preprocess` module.

If we change the postcondition (line 2 of [hw2/benchmark/array_if](#)) with `false`, the verifier should return the following result:

```

1  === Program ===
2  @pre {(true /\ (forall [a]. len[a] >= 0))}
3  @post {false}
4  ...
5  === Verification Result ===
6  false, 0
7  Time : ...

```

which states that the function specification was not proven (`false` at line 6) and thus the verifier did not attempt to validate the safety of each assertion (0 at line 6) – that is, each assertion's safety is verified only when a program is proven to be partially correct with respect to its precondition and postcondition.

Similarly, given [hw2/benchmark/simple_loop](#), you should obtain the result below:

```

1  ...
2  [1] line 20 : safe
3  [2] line 25 : safe
4  [3] line 26 : safe
5  [4] line 27 : safe
6  [5] line 28 : safe
7  [6] line 29 : potentially unsafe
8  === Verification Result ===
9  true, 5
10 Time : ...

```

3 Structure of the Project

You can find the following files in the directory [hw2/code](#).

- `verify.ml`: **Your job is to complete and submit this file only. Please do not modify the file name.**
- `main.ml`: contains the driver code that invokes the main routine (run from `verify.ml`).
- `lang.ml`: contains the definition of our target language. A program consists of a 6-tuple:

$$(pre, post, fid, \vec{i}, o, c)$$

¹<https://github.com/gist-pal/ec4219-software-engineering/blob/main/homework/hw2/code/preprocess.ml#L139-L148> – you do not need to understand the implementation details. Just be aware that the constraint is automatically added to a function's precondition.

where pre and $post$ are FOL formulas, fid is a function identifier (name), $\vec{i}$ is a sequence of input parameters with type declarations, o is an output parameter, and c is a command. The command c is defined with l-values (lv) and expressions (e).

$$\begin{aligned} lv &\rightarrow x \mid x[e] \\ e &\rightarrow n \mid \mathbf{len}(x) \mid lv \mid e_1 + e_2 \mid e_1 - e_2 \mid e_1 * e_2 \\ &\quad \mid \mathbf{true} \mid \mathbf{false} \mid e_1 \prec e_2 \mid \neg e \mid e_1 \&\& e_2 \mid e_1 \parallel e_2 \\ c &\rightarrow \mathbf{decl} \ x \mid lv = e \mid \mathbf{skip} \mid c_1; c_2 \mid \mathbf{if} \ e \ c_1 \ c_2 \mid \mathbf{while} \ [f] \ e \ c \mid \mathbf{assert}(e) \mid \mathbf{return}(e) \end{aligned}$$

where $\prec$ denotes the standard binary comparison operators ($=$, $!$, $<$, $\leq$, $>$, $\geq$). The intended semantics should be clear throughout the following explanations.

- **decl** x : Each variable or element gets assigned default values. The default values for **int** and **bool** are 0 and *false*, respectively. For example, given *int* x , 0 should be assigned to x . As another example, given *int*[10] x , 0 should be assigned to $x[0], \dots, x[9]$.
- $lv = e$: Given an assignment $x = y$ where both x and y are array-typed variables, we assume that all the elements in y are copied into x . That is, the array-typed variables are not aliased in our language. The other cases are standard.
- **while** $[f] \ e \ c$: f represents an annotated loop invariant and it does not have effects on the program semantics; it just helps to perform verification more precisely.
- **assert**(e): **assert** does not affect the program semantics; **assert** just expresses properties expected to hold at certain locations, and nothing happens at runtime even if the conditions in **assert** do not hold.

We will assume that programs to verify are well-typed. For example, if x is an int-typed variable, the command like $\mathbf{len}(x)$ will not appear.

- **term.ml**: implements the definition of terms in FOL. In this module, a bool-typed term is a formula.
- **solver.ml**: implements the interface to invoke the Z3 SMT solver.
- **utils.ml**: provides the helper functions that may be necessary to complete **verify.ml**.
- **preprocess.ml**: performs preprocessing (type resolution and strengthening function preconditions) before the main verification routine. You do not need to understand the details of this module.
- ***.mli** files are the interfaces for the modules implemented in ***.ml** files. Only definitions declared in ***.mli** can be accessed from external modules.
- **parser.mly** and **lexer.mll**: contain the lexer and parser specifications in **ocamllex** and **ocamlyacc**, respectively. You do not need to understand these details for this assignment.

4 How to Build

Make the build script executable by running the command below in the directory **hw1/code**:

```
$ chmod +x build
```

Once you complete `verifier.ml`, you can build the project:

```
$ ./build
```

Then, the executable `./main.native` will be generated. You can run it as follows.

```
$ ./main.native -input YOUR_TESTCODE
```